HO CHI MINH UNIVERSITY
UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

- Programming Algorithm –

Geometry in Competitive programming

Class: 19CLC5

Student: Đỗ Vương Phúc

Contents

Table of figures

1. Induction

Since computer science grow bigger and bigger, mathematics has been applied more and more. One of the special topic we will face in competitive programming, in artificial intelligent and in game design is geometry using algebra. Geometry in computer science requires a high knowledge about mathematics and linear algebra. Before reading this document, you need to have a certain base knowledge about geometry algebra and linear algebra.

In artificial intelligent, we usually place our data on plane and using statistic to calculate them. Others will using geometry to classify, cluster, etc. This document will provide you the basic knowledge about common problems in geometry algebra in computer science.

This document will write using C/C++ programming language to illustrate the algorithm.

2. Represent geometry component in programming

2.1. Point

As we may know that in geometry, the most basic components are line and point. Moreover, with two points, we can establish a segment or a vector. To represent these components, we will create our data type by using struct keyword. Point is the easiest component to be represented, a point we will place it on our plane Oxy by $x-coordinate$ and $y-coordinate$:

```
struct Point {
	double x;
	double y;
};
```

By this way, we can represent a point easily. However, we will meet an inconvenient issue that we cannot set the coordinate while declaring directly. To improve this issue, we will use constructor to declare. We will create 2 constructors, the first one call without parameters (this will initialize our point at O) and the second one will pass in the initial x and initial y:

```
struct Point {
	double x;
	double y;
	Point() {
		x = 0;
		y = 0;
	}
	Point(double iX, double iY) {
		x = iX;
		y = iY;
	}
};
```

2.2. Vector

Next step, we will represent a vector which established from two points. Given two points $A(x_A, y_A)$ and $B(x_B, y_B)$, we will establish our vector $\overrightarrow{AB} = (x_B - x_A, y_B - y_A)$. Following the way we create point structure, we will define as below:

```
struct Vector {
	double x;
	double y;
	Vector() {
		x = 0;
		y = 0;
	}
	Vector(double iX, double iY) {
		x = iX;
		y = iY;
	}
	Vector(Point pA, Point pB) {
		x = pB.x - pA.x;
		y = pB.y - pA.y;
	}
};
```

2.3. Line

Vector $\overrightarrow{AB}$ is a vector which represent for the direction of line AB, we can represent a line AB by an equation $ax + by + c = 0$ with a, b, c are coefficients. Our normal vector of $\overrightarrow{u_{AB}} = (x_u, y_u)$ is $\overrightarrow{n_{AB}} = (x_n, y_n)$ with $x_n x_u + y_n y_u = 0$, it lead to $\overrightarrow{n_{AB}} = (-y_u, x_u) = (y_A - y_B, x_B - x_A)$. The a, b coefficients are coordinates of the normal vector. Then we can easily calculate c coefficient by place the point A into our equation:

$$\begin{cases} a = y_A - y_B \\ b = x_B - x_A \\ c = x_A y_B - x_B y_A \end{cases}$$

```
struct Line {
	double a, b, c;
	Line() {
		a = b = c = 0;
	};
	Line(Point pA, Point pB) {
		a = pA.y - pB.y;
		b = pB.x - pA.x;
		c = pA.x * pB.y - pA.y * pB.x;
	};
};
```

For other shape and component, we can easy establish them from above components or equation. For instance, to represent a circle, we can use a center point and a double variable R, or else we can use an equation $(x-a)^2+(y-b)^2=R^2$ with $I(a,b)$ is the center point to represent.

3. Basic calculation in geometry algebra

3.1. Distance of two points

To get the distance of two points, we will base on the distance of the vertical and the horizontal. Place our points on plane Oxy, we can see that the distance of our points can easily be calculated by Pythagoras theorem for triangle:

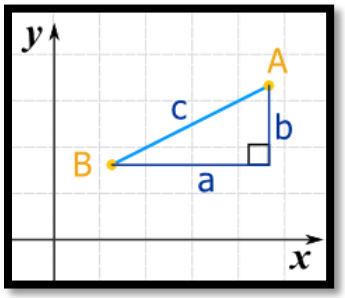

Figure 1. Pythagoras theorem

So our distance can be calculated as: $c=\sqrt{a^2+b^2}$ in term of coordinate, it will be $distance(AB)=\sqrt{(x_A-x_B)^2+(y_A-y_B)^2}$. Also, we can calculate the distance from our vector $\vec{u}=\overrightarrow{AB}$ by $\sqrt{x_u^2+y_u^2}$:

```
double distance(Vector vec) {
	return sqrt(vec.x * vec.x + vec.y * vec.y);
}

double distance(Point pA, Point pB) {
	Vector vec(pA, pB);
	return distance(vec);
}
```

3.2. Intersection of two lines

To get the intersection of two lines, normally we will find the root of two line equations by solving multi-equation. But we cannot do that while programming easily, in this case we will use Carmer's rule in linear algebra. Given our two lines:

$$\begin{cases}(d_1) & a_1x+b_1y=-c_1\\ (d_2) & a_2x+b_2y=-c_2\end{cases}$$

We convert our a,b,c coefficient into matrix, our coefficient will be split into matrix B. Our multi-equations can be represented as $AX=B$ with:

$$A=\begin{pmatrix}a_1 & b_1\\ a_2 & b_2\end{pmatrix}\ X=\begin{pmatrix}x_1\\ x_2\end{pmatrix}\ B=\begin{pmatrix}-c_1\\ -c_2\end{pmatrix}$$

As Cramer said that our root will be calculated from the determinant of our matrix:

$$x_i=\frac{det(A)}{det(A_i)}$$

With A_i is created by replacing our column i^{th} of matrix A by matrix B. Remind that we calculate the determinant of a matrix $2x2$ by: $\begin{vmatrix}a_{11} & a_{12}\\ a_{21} & a_{22}\end{vmatrix}=a_{11}a_{22}-a_{12}a_{21}$. So that we will have three determinants sequentially are:

$$\begin{cases}D & =det(A) & =a_1b_2-a_2b_1\\ Dx & =det(A_1) & =b_1c_2-b_2c_1\\ Dy & =det(A_2) & =c_1a_2-c_2a_1\end{cases}$$

We can see that if $D=Dx=Dy=0$ that means our lines are coincided. Otherwise, if only $D=0$ which means $u_{d1}=u_{d2}$, also means our lines are parallel. Lastly with $D\neq 0$, we will have the unique root which is our intersection:

$$M(\frac{Dx}{D},\frac{Dy}{D})$$

Until now, if you understand how we use Cramer's rule to find the intersection, we can easily write our function as follow. Our function will return an integer which means our two lines are parallel, coincide or intersect. And it will return the coordinate of intersection through reference variable. However because of using double data type, we can compare directly to zero, we need to use an epsilon value which is very small, then compare the absolute value to epsilon:

```
#define e 0.0001

int getIntersect(Line line1, Line line2, double& x, double& y) {
	double D, Dx, Dy;
	D  = line1.a * line2.b - line1.b * line2.a;
	Dx = line1.b * line2.c - line1.c * line2.b;
	Dy = line1.c * line2.a - line1.a * line2.c;

	if (abs(D) < e) {
		if (abs(Dx) < e && abs(Dy) < e) {
			//Coincided
			return 2;
		}
		//Parallel
		return 1;
	}

	//Intersect
	x = Dx / D;
	y = Dy / D;
	return 0;
}
```

3.3. Cos and angle

Thanks to Law of Cosine, now we can calculate the cosine value of an angle from the distance. Law of Cosine said that: In a triangle, calls three side segments sequentially a, b, c and the opposite angle are A, B, C, we will have:

$$\begin{cases} a^2 = b^2 + c^2 - 2bc\cos(A) \\ b^2 = a^2 + c^2 - 2ac\cos(B) \\ c^2 = a^2 + b^2 - 2ab\cos(C) \end{cases}$$

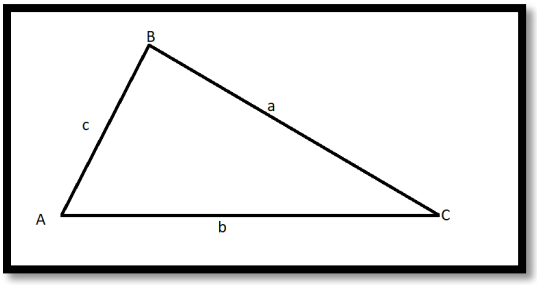

Figure 2. Triangle side segment

So that we can calculate our cosine of angle from 3 points by:

$$\begin{cases} \cos(A) = \dfrac{b^2 + c^2 - a^2}{2bc} \\ \cos(B) = \dfrac{a^2 + c^2 - b^2}{2ac} \\ \cos(C) = \dfrac{a^2 + b^2 - c^2}{2ab} \end{cases}$$

We will summarize into a function which gets in 3 points $p1, p2, p3$ and our function will return the $\cos(p1p2p3)$ in radian:

```
double getCos(Point p1, Point p2, Point p3) {
	double d12, d13, d23;
	d12 = distance(p1, p2);
	d13 = distance(p1, p3);
	d23 = distance(p2, p3);

	double numeral = d12 * d12 + d23 * d23 - d13 * d13;
	double denum = 2 * d12 * d23;
	double result = numeral / denum;

	return result;
}
```

However, we still have one more problem. Law of Cosine can only be applied when we knew the distance, which mean we will need point. So how can we get the cosine of angle between 2 lines or 2 vectors? We will base on the scalar between our two vectors. Given $\vec{a} = (x_1, y_1)$ and $\vec{b} = (x_2, y_2)$:

$$\cos(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}||\vec{b}|} = \frac{x_1 x_2 + y_1 y_2}{\sqrt{x_1^2 + y_1^2}\sqrt{x_2^2 + y_2^2}}$$

With this idea, we need to write a dot function for two vectors calculating their dot production. So that all our source code can be written as following:

```
double getDot(Vector a, Vector b) {
	return a.x * b.x + a.y * b.y;
}

double getCos(Vector a, Vector b) {
	double numeral = getDot(a, b);
	double denum = distance(a) * distance(b);
	double result = numeral / denum;

	return result;
}
```

3.4. Rotation of vector

3.4.1. Rotate detection

In real life, we can represent a movement using a vector. Following that idea, if an animal move on a plane, we can treat movement of the animal as a sequence of vectors. So how can we know whether that animal moves to the left or the right?

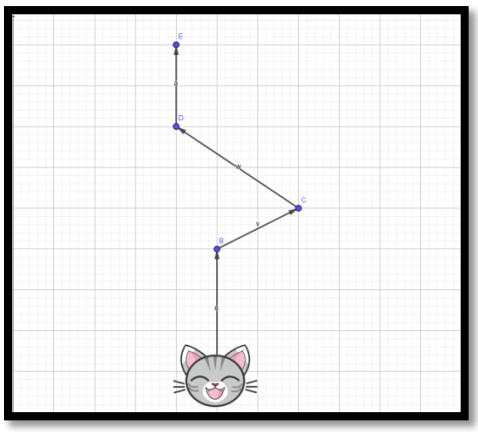

Figure 3. Movement of the cat

As we can see, if our animal move to right, our vector will rotate to right as the same (rotate clockwise) and vice versa. So we will base on the attribute of cross production of two vectors. This production generates a new vector perpendicular to the plane which contains two multiplier vectors. For example, we have 2 vectors on plane Oxy are $\vec{a} = (x_a, y_a, 0)$ and $\vec{b} = (x_b, y_b, 0)$, our cross product $\vec{c} = \vec{a} \times \vec{b}$ which has the coordinate:

$$\vec{c} = \left(\begin{vmatrix} y_a & 0 \\ y_b & 0 \end{vmatrix}, \begin{vmatrix} 0 & x_a \\ 0 & y_a \end{vmatrix}, \begin{vmatrix} x_a & y_a \\ x_b & y_b \end{vmatrix} \right)$$

By this way, the c_z coordinate of $\vec{c}$ will base on the rotation from $\vec{a}$ to $\vec{b}$. If we change the order of production into $\vec{b} \times \vec{a}$, our vector $\vec{c}$ will be reversed on z-axis:

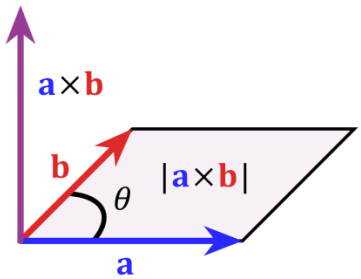

Figure 4. Cross product of vector

So we just need to check c_z value, which means we will use $\begin{vmatrix} x_a & y_a \\ x_b & y_b \end{vmatrix} = x_a y_b - x_b y_a$ value. If $c_z > 0$ that means our vector rotates counter-clockwise (CCW), or else if $c_z < 0$ means that it rotates clockwise (CW). Of course, if $c_z = 0$ means our vector didn't change the direction (go straight):

```
int CCW(Vector a, Vector b) {
	double cz = a.x * b.y - a.y * b.x;

	if (abs(cz) < e) {
		//Unchange direction
		return 0;
	}

	if (cz > 0) {
		//Rotate counter-clockwise
		return 1;
	}

	//Rotate clockwise
	return -1;
}
```

3.4.2. Movement of the weird animal problem

A biology scientist is researching behavior of a weird animal. He has collected a list of point where that animal move to, be that data very big. He cannot classify each step that animal move to left or right by himself, so he ask you to write a program to help him. He will give you a list of point in a plane Oxy contains n point(s). Then your program should output $n-2$ character "L" or "R" or "S" is the direction the animal move from $point_i$ to $point_{i+1}$:

Input	Output
- First line is an integer n with ($n > 2$) - n next line(s) contains 2 numbers are x and y of $point_i$	- Only one line contains string is your solution (string has length is $n-2$)
6 0 0 0 1 1 2 -1 3 0 4 0 5	RLRL

In this problem, we will create an array of point to store all the input. We will combine 3 points into 2 vectors and use our CCW function which I have mention above:

```
#include <iostream>
#define e 0.0001
using namespace std;

struct Point {
	double x;
	double y;
	Point() {}
	Point(double ix, double iy) {
		x = ix;
		y = iy;
	}
};

struct Vector {
	double x;
	double y;
	Vector() {
		x = 0;
		y = 0;
	}
	Vector(double iX, double iY) {
		x = iX;
		y = iY;
	}
	Vector(Point pA, Point pB) {
		x = pB.x - pA.x;
		y = pB.y - pA.y;
	}
};

int CCW(Vector a, Vector b) {
	double cz = a.x * b.y - a.y * b.x;

	if (abs(cz) < e) {
		//Unchange direction
		return 0;
	}

	if (cz > 0) {
		//Rotate counter-clockwise
		return 1;
	}

	//Rotate clockwise
	return -1;
}

int main() {
	//Input
	int n;
	cin >> n;
	Point* a = new Point[n];
	for (int i = 0; i < n; i++) {
		cin >> a[i].x >> a[i].y;
	}
```

```
	//Solution
	string result = "";
	for (int i = 0; i < n - 2; i++) {
		Vector v1(a[i], a[i + 1]);
		Vector v2(a[i + 1], a[i + 2]);
		switch (CCW(v1, v2)) {
		case (-1):
			result += "R";
			break;
		case (0):
			result += "S";
			break;
		case (1):
			result += "L";
			break;
		}
	}

	cout << result;

	//Release memory
	delete[] a;
}
```

4. Triangle problem

4.1. Area of triangle

Not only components in geometry, there are other things we may consider when we work with shapes in geometry. In building architecture, people also considered about the area of a land. Mission of computer scientist is to use computer and science to help everyone solve real life problem. In this time, we will move to calculate the area of a shape. We will start from the easiest one – area of a triangle.

4.1.1. Heron's formula

In normal way, we usually use Heron's formula which is very basic for everyone to use. It said that: Given a triangle with 3 points are A, B, C, the area of that triangle can be calculated equal to $S = \sqrt{p(p-a)(p-b)(p-c)}$ with p is a half of perimeter and a, b, c is the length of side segments.

With this way, we must calculate the perimeter first and calculate the distance of our 3 side segments. There are a lot of calculation we must do to get the area:

```
double areaTriangleHeron(Point A, Point B, Point C) {
	double dAB, dBC, dAC;
	dAB = distance(A, B);
	dBC = distance(B, C);
	dAC = distance(A, C);

	double p = (dAB + dBC + dAC) / 2;
	double area = sqrt(p * (p - dAB) * (p - dBC) * (p - dAC));
	return area;
}
```

4.1.2. Cross product – Linear algebra way

Another way, we can use magnitude of a cross product. In computer science, we may use this way. In term of efficiency, we don't need to call distance function to calculate the area. However, to prove the correctness of this way is more complicated and require more knowledge. We may know that the magnitude of cross production of two vectors is the area of a parallelogram:

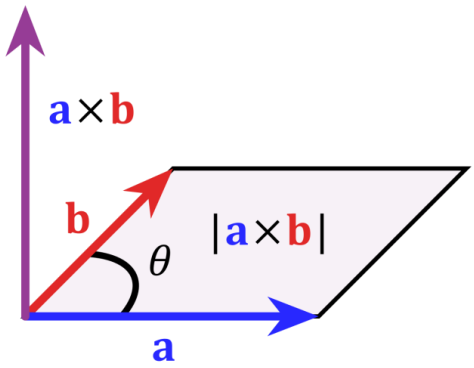

Figure 5. Magnitude of cross product

So we can easily get the area of triangle by divide the magnitude of cross production by 2. Let's treat

$$\begin{cases} \overrightarrow{AB} = \vec{a} = (x_B - x_A, y_B - y_A) = (x_a, y_a) \\ \overrightarrow{AC} = \vec{b} = (x_C - x_A, y_C - y_A) = (x_b, y_b) \end{cases}$$

The same as we have mention in Rotate detection, our $\vec{c} = \vec{a} \times \vec{b} = (0, 0, x_a y_b - x_b y_a)$. That means our magnitude of cross production $|\vec{c}| = |x_a y_b - x_b y_a| = |(x_B - x_A)(y_C - y_A) - (x_C - x_A)(y_B - y_A)|$. Finally, our area formula is:

$$Area_{\triangle} = \frac{1}{2} |(x_B - x_A)(y_C - y_A) - (x_C - x_A)(y_B - y_A)|$$

As we can see, in this solution, we just only base on the coordinate and don't need to calculate the distance:

```
double areaTriangleCross(Point A, Point B, Point C) {
	double area = 0.5 * abs((B.x - A.x) * (C.y - A.y) - (C.x - A.x) * (B.y - A.y));
	return area;
}
```

4.2. Center point of circumscribed circle of a triangle

A circumscribed circle of a triangle is a circle which crosses all three vertices of the triangle. The distances from this center point to any vertex of the triangle are the same (equal to radius of the circle):

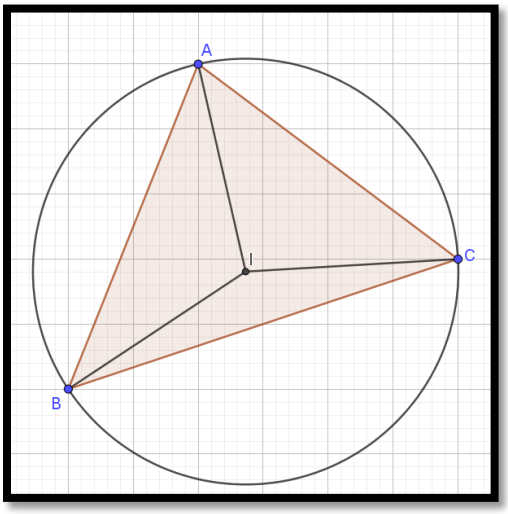

Figure 6. Circumscribed circle of triangle

The center point of circumscribed circle of a triangle is the intersection of two perpendicular bisectors of two side segments:

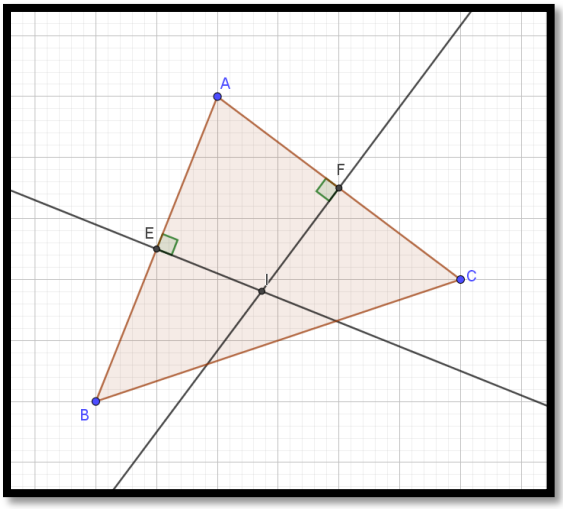

Figure 7. Intersection of two perpendicular bisectors

First we will write a function to get the middle point from two points, easily we just need to calculate average value of total $x-coordinate$ and $y-coordinate$:

```
Point getMiddlePoint(Point A, Point B) {
	Point M;
	M.x = 0.5 * (A.x + B.x);
	M.y = 0.5 * (A.y + B.y);
	return M;
}
```

Of course that we will create a perpendicular bisector which is perpendicular of side segment and go through our middle point:

```
Line getPerpendicularBisector(Point A, Point B) {
	Line AB(A, B);
	Point mid = getMiddlePoint(A, B);
	Line result;

	result.a = -AB.b;
	result.b = AB.a;
	result.c = -(result.a * mid.x + result.b * mid.y);

	return result;
}
```

After we got the perpendicular bisector of two side segments, we easily calculate the intersection by using function above (in Intersection of two lines):

```
Point getCenterCircumscribed(Point A, Point B, Point C) {
	Line bisector1 = getPerpendicularBisector(A, B);
	Line bisector2 = getPerpendicularBisector(B, C);
	Point intersect;

	int result = getIntersect(bisector1, bisector2, intersect.x, intersect.y);
	return intersect;
}
```

4.3. Center point of inscribed circle of a triangle

In contrast, with circumscribed circle, inscribed circle is the largest circle contained in our triangle, and it touches three side segments once per side:

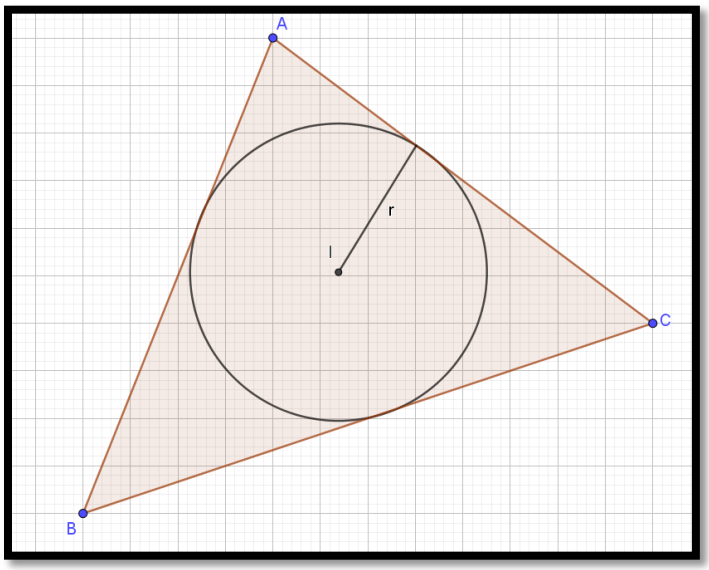

Figure 8. Inscribed circle of triangle

The center point of the inscribed circle is the intersection of two angle bisectors of two side segments. However, extracting the angle bisector is complicated so we will use one property of inscribed circle of a triangle. Calls center point is I we also get equation (Can be proved using property of angle bisector):

$$AB.\overrightarrow{IC} + BC.\overrightarrow{IA} + AC.\overrightarrow{IB} = \vec{0}$$

This means we can easily calculate the coordinate of point I by expanding the formula above:

$$\begin{cases} x_I = \dfrac{AB.x_C + BC.x_A + AC.x_B}{perimeter} \\ y_I = \dfrac{AB.y_C + BC.y_A + AC.y_B}{perimeter} \end{cases}$$

Our program can be written in one go same as following:

```
Point getCenterInscribed(Point A, Point B, Point C) {
	double dAB, dBC, dAC;
	dAB = distance(A, B);
	dBC = distance(B, C);
	dAC = distance(A, C);
	double perimeter = dAB + dBC + dAC;

	Point intersect;
	intersect.x = (A.x * dBC + B.x * dAC + C.x * dAB) / (perimeter);
	intersect.y = (A.y * dBC + B.y * dAC + C.y * dAB) / (perimeter);

	return intersect;
}
```

5. Polygon

5.1. Represent a polygon

To represent a polygon, we will create a set of n points called a which $a_i a_{i+1}$ is a segment (with $i < n-1$) and $a_{n-1}a_0$ is the last segment. Our structure will contain an integer n is the number of points in our polygon and an array of points. Additionally, we will have a constructor to initialize a polygon of n points and a method to release our array after used:

```
struct Poly {
	int n = 0;
	Point* a = nullptr;
	Poly() {
		n = 0;
	}
	Poly(int iN) {
		n = iN;
		a = new Point[n];
	}
	void release() {
		delete[] a;
	}
};
```

5.2. Perimeter of a polygon

Calculating perimeter sound very easy, but with a polygon, we have a lot of segments so we cannot calculate the distance one by one easily. As we mention above, our polygon will be represented by an array of points which $p_i p_{i+1}$ is one segment. So for calculate the perimeter for polygon n points, we will add one more point has same coordinate with the first point at the end. Our perimeter can be calculate by:

$$P = \sum_{0}^{n-1} p_i p_{i+1}$$

Our function will be same as below:

```
double perimeter(Poly p) {
	double result = 0;

	//Create list of points
	Point* lst = new Point[p.n+1];
	for (int i = 0; i < p.n; i++) {
		lst[i] = p.a[i];
	}
	lst[p.n] = lst[0];

	//Calculate perimeter
	for (int i = 0; i < p.n; i++) {
		result += distance(lst[i], lst[i + 1]);
	}

	//Release memory
	delete[] lst;

	return result;
}
```

5.3. Convex polygon

A convex polygon is a simple polygon (not self-intersecting) in which no line segment between two points on the boundary ever goes outside the polygon. Convex polygon has some properties:

- Every internal angle is strictly less than 180 degrees.
- Every point on every line segment between two points inside or on the boundary of the polygon remains inside or on the boundary.
- The polygon is entirely contained in a closed half-plane defined by each of its edges.
- For each edge, the interior points are all on the same side of the line that the edge defines.
- The angle at each vertex contains all other vertices in its edges and interior.
- The polygon is the convex hull of its edges.

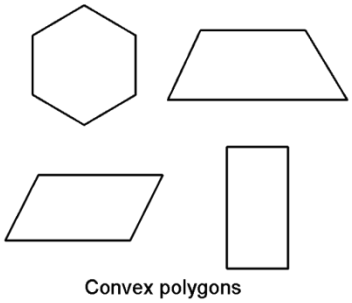

Figure 9. Convex polygon

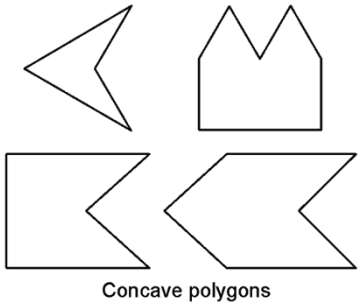

Figure 10. Concave polygon

5.4. Convex hull

Given a set of points, a convex hull or convex envelope is a smallest (in term of number of point) set of points whose points are chosen from given set that contain can contain all the given points. For example, in the picture below we have such a set of point, our blue line is the convex hull which combines by 6 points and it contains all the given points.

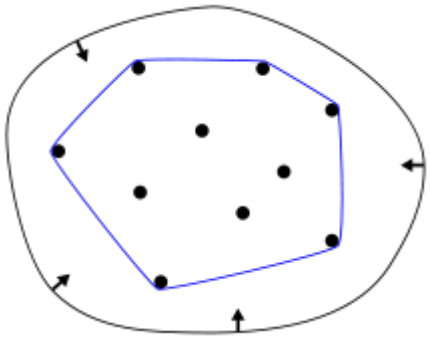

Figure 11. Convex hull

In real life, we have many problems which we can use the convex hull to solve. For instance:

- On a field, there are many sheep. You have to find a way to wrap them all which cost smallest money.
- Placing a lot of nails on a table, choose the way to wrap all the nails so that no one will be in danger.

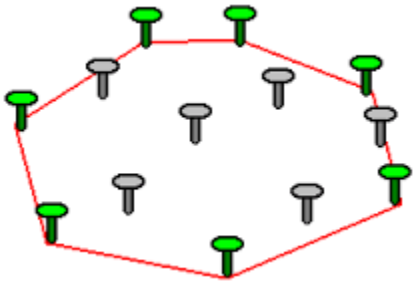

Figure 12. Wrap nails

With humans' mind, we can easily find out the way to wrap our set. But in computer, how can we teach our machine to find out the solution? Let's jump into three algorithms to find the convex hull.

5.4.1. Wrap gift algorithm

The easiest algorithm is the wrap gift algorithm. This algorithm base on how our mind worked, we will follow how we wrap our gift with a ribbon. Firstly, we will choose a point that we sure it will in our convex set then we continuously wrap around. We will represent our algorithm step by step:

1. We will choose a point P which has smallest $y-coordinate$ (if we have many points, we will choose the one has smallest $x-coordinate$). We will sure that P is the point must be chosen.
2. We will create a vector $\vec{u}=(-1,0)$ direct to the left of (Oxy) plane. With this vector, we will rotate clockwise until we meet the first point. That point will be called Q. To do this we will choose the point which $\overrightarrow{PQ},\vec{u}$ is the smallest one, which means $\cos(\overrightarrow{PQ},\vec{u})$ is largest.
3. Assign $\vec{u}=\overrightarrow{PQ}$ and $P=Q$.
4. We will continuously do from step 2 until we get to the first point P_0.

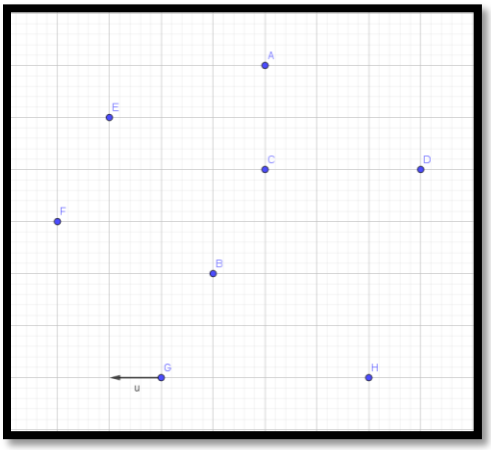

Figure 13. Starting wrapping point

For example, with the set of points above, we will choose G as a starting point then $\vec{u}$ will look to the left. Then the next point Q must be F and our $\vec{u}=\overrightarrow{GF}$:

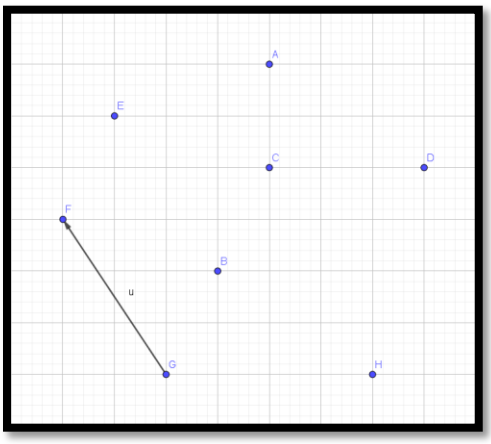

Figure 14. Wrap the first point

We will write a function that returns a Polygon structure. Our solution can be written as follow:

```
#define infty 99999999

Poly getConvexHullWrap(Point* a, int nPoint) {
	//Init convex set of points
	Poly result;
	result.n = 0;
	result.a = new Point[nPoint];

	//Find starting point P zero
	int indexP0 = 0;
	for (int i = 1; i < nPoint; i++) {
		if (a[indexP0].y > a[i].y ||
			(a[indexP0].y == a[i].y && a[indexP0].x > a[i].x)) {
			indexP0 = i;
		}
	}

	//Starting variable
	Vector u(-1,0);
	int indexP;
	indexP = indexP0;

	//Run until P is P0
	do {
		double maxCos = -infty;
		int indexQ = -1;

		//Each point not current P find maximum cos
		for (int i = 0; i < nPoint; i++) {
			if (i != indexP) {
				//Calculate cos value of PQ and u
				double dCos = cos(u, Vector(a[indexP], a[i]));
				if (maxCos < dCos) {
					maxCos = dCos;
					indexQ = i;
				}
			}
		}
		result.a[result.n++] = a[indexP];

		//Assign new vector u and P
		u = Vector(a[indexP], a[indexQ]);
		indexP = indexQ;
	} while (indexP != indexP0);

	return result;
}
```

5.4.2. Graham algorithm

Another algorithm which cost less time complexity but more complicated is Graham's algorithm. This algorithm will base on how we sort our set and the rotation of vector:

1. Same as Wrap gift algorithm, we will find the starting point called O which has smallest $y-coordinate$ (if there are many, choose the smallest $x-coordinate$ one).
2. Ascending sort our set of points by angle of $x-axis$ and $\overrightarrow{OI}$ with I is our rest points (also means descending by cosine value). For example, with the above test-case, our list will be: $O \equiv G$ and our list will be $\{G, H, D, B, C, A, E, F\}$. This also mean, we will descending sort by cosine value:

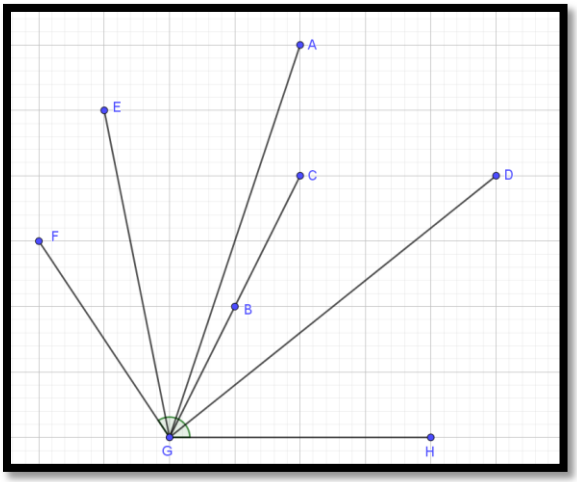

Figure 15. Graham angle sorting

3. Call our solution is polygon K. With each point in sorted set, the current point will be P and number of points in K is i:
   a. Add P to the last of K.
   b. If i is less than 3, we will continue to check the next point in our set. Otherwise, do step c.
   c. If our vector $\overrightarrow{K_{i-2}K_{i-1}}$ rotate to vector $\overrightarrow{K_{i-1}K_i}$ is clockwise rotation, so that mean we must remove K_{i-1} from our set. Because angle $K_{i-2}K_{i-1}K_i$ will create a concave polygon. We will check this continuously until our vector is not rotate to the right or i is less than 4.

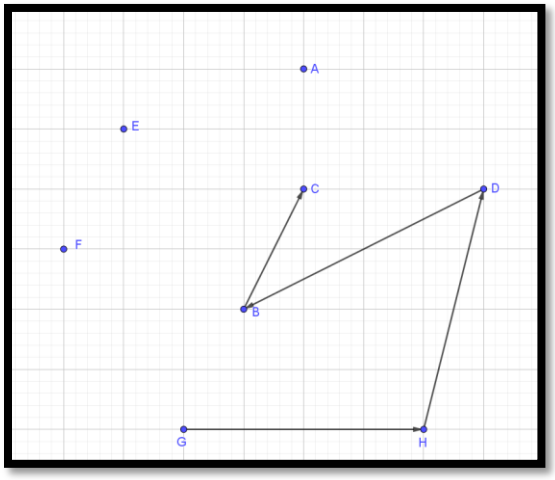

Figure 16. Graham algorithm at 5th step

With the example we mentioned above, at the 5^{th} step, we can see that our triangle BCD will create a concave polygon. This will lead to point B will be removed. First, we will write the sorting function base on quick sort algorithm:

```
//Quick sort
void sort(Point* a, int l, int r) {
	int i=l, j=r;
	Point pivot = a[(l + r) / 2];

	while (i <= j) {
		//Vector direct Ox and vector OPivot
		Vector Ox(1, 0);
		Vector OPivot(a[0], pivot);

		//Cos of two above vector
		double cosOxPivot = cos(Ox, OPivot);

		while (cos(Ox, Vector(a[0], a[i])) > cosOxPivot) i++;
		while (cos(Ox, Vector(a[0], a[j])) < cosOxPivot) j--;

		if (i <= j) {
			swap(a[i++], a[j--]);
		}
	}

	if (i < r) sort(a, i, r);
	if (l < j) sort(a, l, j);
}
```

There are some notes we need to consider. Firstly, at the sorting step, we will not sort the O point. Secondly, our polygon counts from 0 to $n-1$. Lastly, because of checking rotation continuously, we will rewrite our CCW (counter-clockwise) function for three points A, B, C:

```
int CCW(Point A, Point B, Point C) {
	Vector u(A, B);
	Vector v(B, C);
	return CCW(u, v);
}

Poly getConvexHullGraham(Point* a, int nPoint) {
	//Init convex set of points
	Poly result;
	result.n = 0;
	result.a = new Point[nPoint];

	//Find starting point
	int id = 0;
	for (int i = 1; i < nPoint; i++) {
		if (a[i].y < a[id].y ||
			(a[i].y == a[id].y && a[i].x < a[id].x)) {
			id = i;
		}
	}
	swap(a[id], a[0]);

	//Sort our set of point (except 0)
	sort(a, 1, nPoint - 1);

	for (int i = 0; i < nPoint; i++) {
		//Add point
		result.a[result.n++] = a[i];

		if (result.n < 3) {
			continue;
		}

		//Check rotation
		while (result.n>3 &&
			CCW(result.a[result.n - 3],
				result.a[result.n - 2],
				result.a[result.n - 1]) < 0) {
			result.n--;
			result.a[result.n-1] = result.a[result.n];
		}
	}

	return result;
}
```

5.4.3. Monotone chain

Monotone chain is the last algorithm to find the convex hull of a set. This algorithm is an algorithm which optimized from Graham algorithm. In term of efficiency, Graham algorithm is time-consuming because of sorting by cosine value. With monotone chain, we will sort our point base on the coordinate.

This algorithm will find the convex hull of upper chain and lower chain then combine these two chain, we will have a fully convex hull. It will check the rotation of vector as same as Graham algorithm for the upper and lower chain parallel. Monotone chain algorithm can be represented as:

1. Sorting our point ascending by $x-coordinate$ (If there are many, we will sort ascending by $y-coordinate$). This is why monotone chain more effective than Graham algorithm. For example, with our above test case, our sorted set of points must be: $\{F, E, G, B, C, A, H, D\}$

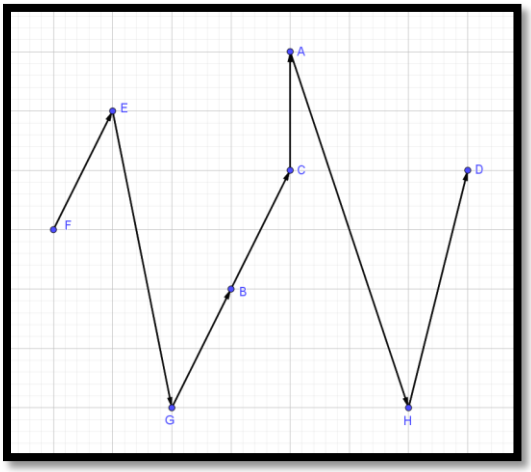

Figure 17. Monotone chain sorting

2. Call our left-most point is P_L (is F in this case), right-most point is P_R (is D in this case). Our both chains will start with P_L.
3. With each point P_i in sorted set (exclusive point P_L).
    a. We will check whether it is in upper chain ($P_L P_i P_R$ is rotated clockwise) or lower chain ($P_L P_i P_R$ is rotate counter-clockwise) then add P_i to our chain. Note that our last point P_R will be at both chains.
    b. Same as Graham algorithm, after add the P_i point into our chain, we will check whether chain create a concave polygon. If our new chain creates an concave polygon, we will remove the previous point of current chain.

For instance, at the 2^{nd} step, point E will be added to the upper chain and point G will be added to the lower chain (Because of FED is rotated clockwise and FGD is rotated counter-clockwise):

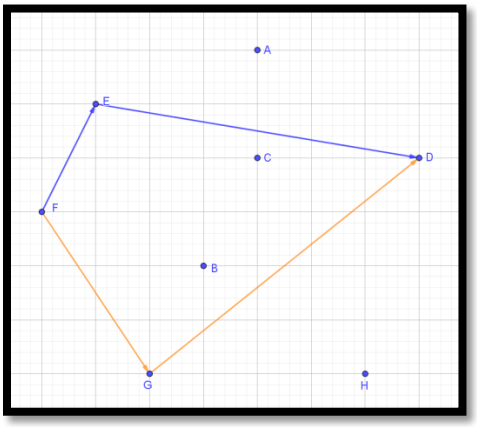

Figure 18. Monotone chain at 2nd step

Same as that, point B, H will be at the lower chain and A, C will be at the upper chain. After running through all the point, our chains must be:

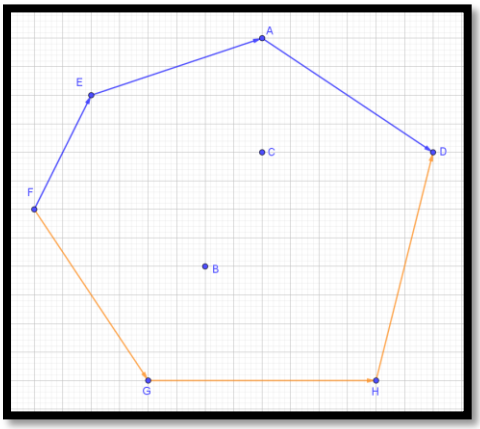

Figure 19. Mono chain upper and lower chain

Lastly, we can easily merge these two chains into a convex hull. Let's head first into our sorting function, this sort function still base on quicksort algorithm, we will pass in a function pointer to compare two points by coordinate:

```
bool cmp(Point A, Point B) {
	return ((A.x < B.x) || (A.x == B.x && A.y < B.y));
}

void sort(Point a[], int l, int r, bool (*cmp)(Point, Point)) {
	int i = l, j = r;
	Point pivot = a[(l + r) / 2];

	while (i <= j) {
		while (cmp(a[i],pivot)) i++;
		while (cmp(pivot,a[j])) j--;

		if (i <= j) {
			swap(a[i++], a[j--]);
		}
	}

	if (i < r) sort(a, i, r, cmp);
	if (l < j) sort(a, l, j, cmp);
}
```

Remember that our starting point of both chains is P_L and the ending point is P_R so that we merge the lower chain with the last point backward to the upper chain:

```
Poly getConvexHullMonotone(Point* a, int nPoint) {
	//Init convex set of points
	Poly result(nPoint); result.n = 0;

	//Sort by coordinate
	sort(a, 0, nPoint - 1, cmp);

	//Left-most, right-most point
	//Upper and lower chain
	Point PL = a[0], PR = a[nPoint - 1];
	Poly upper(nPoint);
	Poly lower(nPoint);
	upper.a[0] = lower.a[0] = PL;
	upper.n = lower.n = 1;

	//With each point (excluisve PL)
	for (int i = 1; i < nPoint; i++) {
		//Upper chain
		if (i == nPoint - 1 || CCW(PL, a[i], PR) == -1) {
			upper.a[upper.n++] = a[i];
			while (upper.n > 3 &&
				CCW(upper.a[upper.n - 3],
					upper.a[upper.n - 2],
					upper.a[upper.n - 1]) != -1) {
				upper.n--;
				upper.a[upper.n - 1] = upper.a[upper.n];
```

```
                        }
                }

                //Lower chain
                if (i == nPoint - 1 || CCW(PL, a[i], PR) == 1) {
                        lower.a[lower.n++] = a[i];
                        while (lower.n > 3 &&
                                CCW(lower.a[lower.n - 3],
                                        lower.a[lower.n - 2],
                                        lower.a[lower.n - 1]) != 1) {
                                lower.n--;
                                lower.a[lower.n - 1] = lower.a[lower.n];
                        }
                }
        }

        //Merge
        for (int i = 0; i < upper.n; i++) {
                result.a[result.n++] = upper.a[i];
        }
        for (int i = lower.n - 2; i > 0; i--) {
                result.a[result.n++] = lower.a[i];
        }

        //Release memory
        upper.release(); lower.release();

        return result;
}
```

5.5. Area of a convex polygon

With convex polygon, we can calculate the area of it easily by splitting it into many triangles. Because of convex polygon properties, if we choose a constant point and two points which sit next to each other, we can create many non-overlapped triangles:

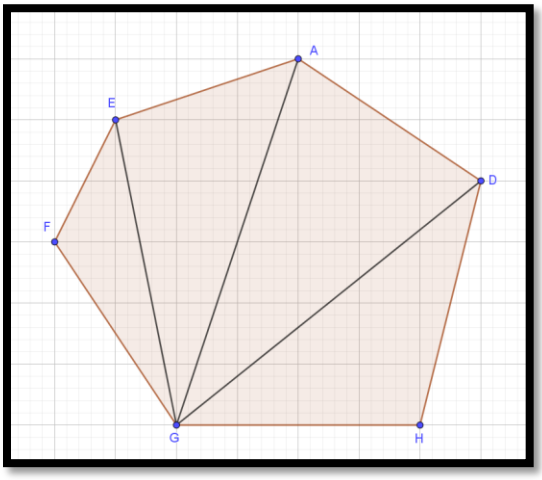

Figure 20. Convex polygon is split into many triangles

With this property, we can easily calculate the area of convex polygon by using area of a triangle which we have mentioned in Area of triangle section:

```
double areaConvexPolygon(Poly p) {
	double area = 0;
	Point P0 = p.a[0];

	for (int i = 1; i < p.n - 1; i++) {
		area += areaTriangleCross(P0, p.a[i], p.a[i + 1]);
	}

	return area;
}
```

5.6. Relative of a point and a polygon

5.6.1. Area method for convex polygon

We can easily prove that if a point is inside a convex polygon, connect that point with all points of the polygon, it will split our polygon into triangles which total area is the area of our convex polygon. In the other word, if a point inside a convex polygon, total area of triangles which split from that point equal area of polygon. With this property, we can check that whether a point is inside or outside a convex polygon by calculating the area.

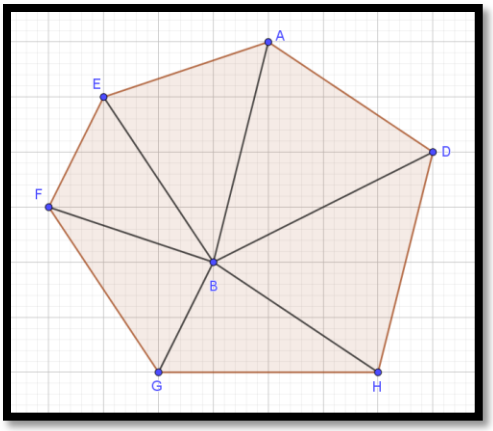

Figure 21. Point inside convex polygon

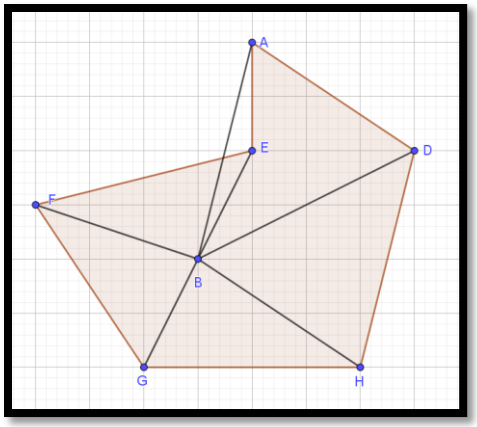

Figure 22. Point inside concave polygon

To calculate the area we will re-use *areaConvexPolygon* and *areaTriangleCross*. First, we also create an array of points to represent our polygon with the last point is same as the first one:

```
#define e 0.00001

bool isInsideConvex(Point point, Poly poly) {
	//Area of polygon
	double polyArea = areaConvexPolygon(poly);

	//Area of triangles
	double sumTriangleArea = 0;
	int nPoint = poly.n + 1;
	Point* a = new Point[nPoint];

	//Copy polygon into array
	for (int i = 0; i < nPoint - 1; i++) {
		a[i] = poly.a[i];
	}
	a[nPoint - 1] = a[0];

	//Calculate sum of triangles
	for (int i = 0; i < nPoint - 1; i++) {
		sumTriangleArea += areaTriangleCross(point, a[i], a[i + 1]);
	}

	//Release memory
	delete[] a;

	return (abs(polyArea - sumTriangleArea) < e);
}
```

5.6.2. Raycast algorithm

So how can we check whether a point is inside or outside a concave polygon? With area method we mentioned above, we cannot apply for the concave polygon because two areas are not the same. This will lead to another algorithm call "Raycast algorithm" or also named "Crossing-number algorithm". From the point we need to check, create a ray into any direction and count numbers of intersection:

- Number of intersection is odd means our point is inside.
- Number of intersection is even means our point is outside.

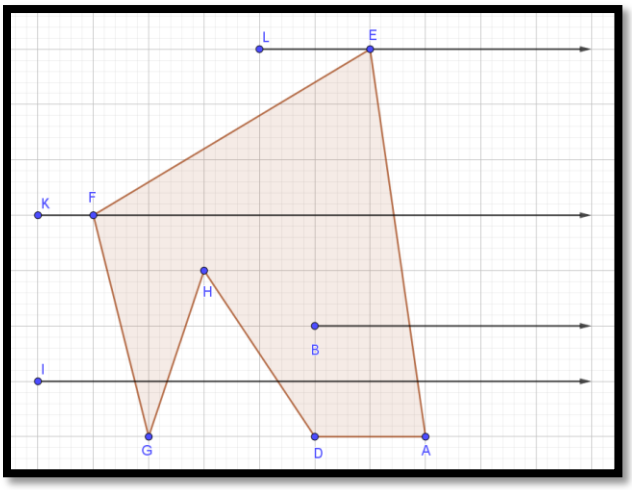

Figure 23. Raycast to the right in case of concave polygon

For easier to do, we will check a ray from checking point to the right. Besides, there are some disadvantageous. First, with the example above, the ray from point L cross only one time even it's not inside. Second, this algorithm cannot be used when the ray is coincided a segment of the polygon:

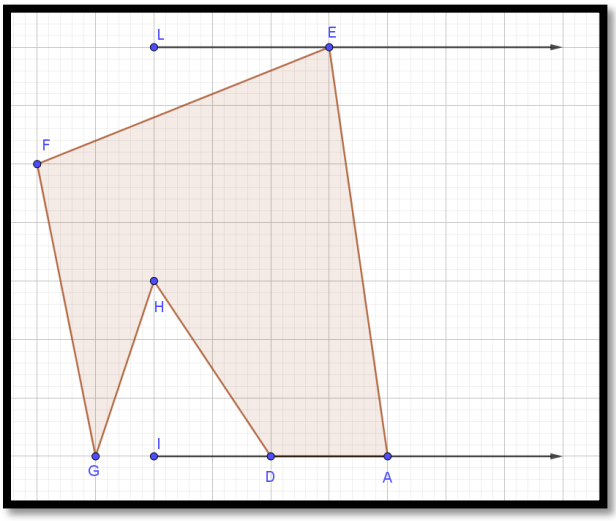

Figure 24. Disadvantageous of raycast algorithm

To count number of intersection of our ray, with each segment of the polygon, we will check $y-coordinate$ of our point is in range from the lower point of current segment to the upper point (exclusive the lower point). For example, with segment EA, we just only check the segment and point E. This also means:

$$\begin{bmatrix} y_E < y_{point} \le y_A \\ y_A < y_{point} \le y_E \end{bmatrix}$$

In other word, we can represent as:

$$\begin{bmatrix} (y_{point} > y_E) \wedge !(y_{point} > y_A) \\ (y_{point} > y_A) \wedge !(y_{point} > y_E) \end{bmatrix}$$

After checking $y-coordinate$, we will check the $x-coordinate$ of our point and the intersection of our ray and the segment. Call our intersection is I and our segment is AB. Our ray can be represent as a line with equation is $y = y_{point}$. Intersection I is a point on line AB whose $y-coordinate$ is $point_y$ and $x-coordinate$ is calculate from line equation: $ax_I + by_I + c = 0$ with $y_I = y_{point}$. So our x_I is:

$$x_I = -\frac{c + by_{point}}{a}$$

Because the ray direct to the right, our point must be $x_{point} < x_I$. This solution can be represented as:

```
bool isInPolygon(Point point, Poly poly) {
	bool result = false;

	//Array of point for the poly
	int nPoint = poly.n + 1;
	Point* a = new Point[nPoint];

	//Copy the poly to array
	for (int i = 0; i < nPoint-1; i++) {
		a[i] = poly.a[i];
	}
	a[nPoint-1] = a[0];

	//Each point
	for (int i = 0; i < poly.n - 1; i++) {
		//y-coordinate
		if (point.y>a[i].y != point.y>a[i+1].y) {
			//Create line
			Line AB(a[i], a[i + 1]);
			if (point.x < (-(AB.c + AB.b * point.y) / AB.a)) {
				result = !result;
			}
		}
	}

	//Release memory
	delete[] a;

	return result;
}
```

6. Conclusion

With this new topic, there are many and many more complicated problem. This is just the basic one to introduce everyone to a new kind of idea in computer science. To solve a problem, we have many ways to do, however each way still has disadvantageous. Geometry in computer science is important for who are planning to study game developing and artificial intelligent.

Hopefully that after this document, you have studied a lot and found that computer science is huge and wide. There are many more topics that you need to study if you want to master.